المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⵓⵙⵙⵏⴰⵡⵉⵏ ⵜⵉⵏⵎⵍⴰⵍⵉⵏ - ⴰⵎⵜ ⵎⵉⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

COMPTE RENDU DE TRAVAUX PRATIQUE PYTHON

1ère Année – Filière Intelligence Artificielle et Cybersécurité
Année Universitaire 2025–2026

Travail réalisé par : Achraf Hamouda

Sous l'encadrement de : Mme. Ibtissame Aouraghe

24 OCTOBRE 2025

ÉCOLE NATIONALE DES SCIENCES APPLIQUEES DE BENI MELLAL
Université Sultan Moulay Slimane

Table des matières

1ère Année – Filière Intelligence Artificielle et Cybersécurité Année Universitaire2025–2026 0

Introduction 2

Exercice 1 : Jeu de devinettes 3

Exécution1 : 4

Exercice 2 : Nombre Armstrong (ou narcissique) 6

Exécution2 : 7

Exercice 3 : Vérification de palindrome sans chaînes 8

Exécution3 : 9

Exercice 4 : Calcul de la racine carrée par dichotomie 10

Exécution4 : 11

Exercice 5 : Nombre parfait le plus proche 12

Exécution5 : 13

Introduction

Ce TP s'inscrit dans le cadre du module de programmation Python, dont l'objectif est de nous initier à la logique algorithmique à travers un langage moderne, lisible et puissant, largement utilisé dans les domaines de l'intelligence artificielle, de la cybersécurité et de la science des données.

Après le premier TP consacré à l'installation de l'environnement de travail et à la découverte des interfaces Spyder et Jupyter Notebook, nous avons exécuté nos premières commandes dans un langage fascinant : Python, conçu par Guido van Rossum.

Dans ce deuxième TP, nous abordons les bases de la programmation en Python, notamment la manipulation des variables, des opérateurs et des structures conditionnelles.

Ce travail constitue une première immersion dans la logique algorithmique à travers Python, en continuité avec les notions vues en algorithmique et en langage C durant la classe préparatoire.

Nous avons utilisé Spyder (Python 3.13) pour écrire, exécuter et analyser nos programmes. Ce TP marque donc le début de notre apprentissage pratique de la programmation et illustre la mise en œuvre d'algorithmes fondamentaux dans un environnement moderne et intuitif.

Jeu de devinettes

Exercice 1 : Jeu de devinettes

Dans cette exercice l'ordinateur choisit un nombre aléatoire entre 1 et 100, et l'utilisateur doit le deviner. Cet exercice introduit la notion de boucles, conditions et importation de modules (random).

```
temp.py* ×
import random
a=random.randint(1, 100)
print(a)
b=int(input("Entrer le nombre a deviner : "))
tentative=10
while tentative>0 :
    if a==b:
      print("CORRECT, Vous êtes HEROS")
      break
    else:
      tentative -=1
      if tentative==0 :
          print("Vous avez échoué, Réessayez plus tard")
          break
      if b>a : print("Le nombre que vous avez entré est grand!")
      else :
               print("Le nombre que vous avez entré est petit!")
               print("Réessayez")
               print(f"Vous avez encore seulement {tentative} Tentative")
               print("---------------------------------------------")
               b=int(input("Entrer le nombre a deviner : "))
```

Figure 1 : code source

- Dans un premier temps, on **importe le module** *random* qui contient plusieurs fonctions, dont *randint()*.
- Ensuite, on **demande à l'utilisateur d'entrer une valeur** à deviner avec *int(input(...))* pour récupérer un entier.
- On utilise une **variable** *tentative* pour représenter le **nombre de chances restantes**, initialisée à la valeur maximale indiquée dans le TP.
- Puis on utilise la boucle *while tentative >0 :* pour à chaque fois on ne dépasse pas les 10 tentatives possibles.
- Puis la 1ere if vérifie si le nombre entrée par utilisateurs est égale à la valeur à deviner sinon(else) on soustraire 1 de la valeur courante pour indiquer que c'est fausse et cette tentative est consommée, puis on utilise des *print()* pour orienter l'utilisateur, ensuit on

met une autre *input()* pour entrer à nouveau la valeur à deviner à la fin du else, si la 2eme valeur entrée par l'utilisateur est encore fausse while trouve que *tentative =8* car la 1ere valeur est encore fausse donc après qu'on sortir de while *tentative =9* et ainsi de suite…

- **Enfin**, si toutes les *tentatives* sont consommées, on **affiche un message d'échec**.

Exécution1 :

```
In [33]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
44
Entrer le nombre a deviner : 44
CORRECT, Vous êtes HEROS
```

Figure 2 : Cas de réussite ; valeur correcte trouvée

```
In [31]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
30
Entrer le nombre a deviner : 10
Le nombre que vous avez entré est petit!
Réessayez
Vous avez encore seulement 9 Tentative
--------------------------------------------
Entrer le nombre a deviner : 12
Le nombre que vous avez entré est petit!
Réessayez
Vous avez encore seulement 8 Tentative
--------------------------------------------
Entrer le nombre a deviner : 19
Le nombre que vous avez entré est petit!
Réessayez
Vous avez encore seulement 7 Tentative
--------------------------------------------
Entrer le nombre a deviner : 34
Le nombre que vous avez entré est grand!
```

```
Réessayez
Vous avez encore seulement 6 Tentative
--------------------------------------------
Entrer le nombre a deviner : 33
Le nombre que vous avez entré est grand!
Réessayez
Vous avez encore seulement 5 Tentative
--------------------------------------------
Entrer le nombre a deviner : 31
Le nombre que vous avez entré est grand!
Réessayez
Vous avez encore seulement 4 Tentative
--------------------------------------------
Entrer le nombre a deviner : 28
Le nombre que vous avez entré est petit!
Réessayez
Vous avez encore seulement 3 Tentative
--------------------------------------------
Entrer le nombre a deviner : 1
Le nombre que vous avez entré est petit!
```

```
Vous avez encore seulement 4 Tentative
--------------------------------------------
Entrer le nombre a deviner : 28
Le nombre que vous avez entré est petit!
Réessayez
Vous avez encore seulement 3 Tentative
--------------------------------------------
Entrer le nombre a deviner : 1
Le nombre que vous avez entré est petit!
Réessayez
Vous avez encore seulement 2 Tentative
--------------------------------------------
Entrer le nombre a deviner : 1
Le nombre que vous avez entré est petit!
Réessayez
Vous avez encore seulement 1 Tentative
--------------------------------------------
Entrer le nombre a deviner : 122
Vous avez échoué, Réessayez plus tard
```

Figure 3,4 et 5 : Cas d'échec ; essais incorrects successifs jusqu'à la fin des tentatives.

Nombre Armstrong (ou narcissique)

Exercice 2 : Nombre Armstrong (ou narcissique)

Dans cet exercice, nous allons écrire un programme dans notre éditeur Python(Spyder) qui vérifie si un nombre entré à trois chiffres est un nombre Armstrong ou non.

Pour cela, nous utiliserons une boucle for ainsi que des conditions if et else.

```
# TP2 – Python : Vérification d'un nombre Armstrong
# Nom : HAMOUDA
# Prénom : Achraf
# Numéro Apogée : 23010848

a = input("Entrer votre chiffre : ")
b = len(a)
somme = 0
for c in a :
    somme += int(c) **b
if somme == int(a) :
    print("Votre nombre est Armstrong")
else:
    print("Votre nombre n'est pas Armstrong")
```

- La première chose qu'on a fait est de demander à l'utilisateur d'entre un nombre pour faire le test si il est Armstrong ou non , on a laisser le nombre str pour bien utiliser la fonction len().

- Ensuite, après avoir obtenu la taille du nombre saisi, nous avons initialisé une variable somme à 0, qui représente la somme des puissances des chiffres du nombre entré par l'utilisateur.
- Après, on a fait la boucle for qui parcourt tous les éléments de la chaine de caractère entrer par l'utilisateur on capte chaque éléments par int(c) et après on fait la somme des puissances(len(a)) pour chaque chiffre.
- Enfin, nous avons utilisé une condition if pour vérifier si la somme calculée est égale au nombre original (somme == int(a)).

  Si c'est le cas, le nombre est Armstrong, sinon (else), il ne l'est pas.

Exécution2 :

```
In [6]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
Entrer votre chiffre : 153
Votre nombre est Armstrong

In [7]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
Entrer votre chiffre : 111111
Votre nombre n'est pas Armstrong
```

Figure 4 : Cas de réussite et d'échec.

Vérification de palindrome sans chaînes

Exercice 3 : Vérification de palindrome sans chaînes

Dans cet exercice, nous allons écrire un programme Python qui permet de vérifier si un nombre entier est un palindrome, c'est-à-dire qu'il se lit de la même façon de gauche à droite et de droite à gauche, sans utiliser de chaînes de caractères.

```
# TP2 – Python : Vérification d'un nombre palindrome
# Nom : HAMOUDA
# Prénom : Achraf
# Numéro Apogée : 23010848

a = int(input("Entrez votre nombre : "))
nbr_ancien = a
nbr_inverse = 0

while a > 0:
    chiffre = a % 10
    nbr_inverse = nbr_inverse * 10 + chiffre
    a //= 10

if nbr_ancien == nbr_inverse :
    print("Votre nombre est un palindrome.")
else:
    print("Votre nombre n'est pas un palindrome.")
```

Figure 5: code source

- La première chose quon a faite est de demander à l'utilisateur d'entrer un nombre entier(a).
  Ensuite, on garde ce nombre original dans une autre variable nbr_ancien pour le comparer plus tard.

- Après, on a initialisé une variable nbr_inverse à 0, qui va contenir le nombre inversé à la fin du programme.

- Ensuite, on a créé une boucle while a > 0 qui va tourner tant que le nombre est supérieur à 0.Dans cette boucle, on utilise a % 10 pour récupérer le dernier chiffre du nombre et a //= 10 pour supprimer ce dernier chiffre à chaque tour.
- À chaque itération, on multiplie nbr_inverse par 10 et on ajoute le chiffre extrait, ce qui permet de construire le nombre inversé petit à petit.

- À la fin, on fait une condition if pour comparer le nombre original (nbr_ancien) avec le nombre inversé (nbr_inverse).
  Si les deux sont égaux, le nombre est palindrome, sinon il ne l'est pas.

Exécution3 :

```
In [1]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
Entrez votre nombre : 111
Votre nombre est un palindrome.

In [2]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
Entrez votre nombre : 122
Votre nombre n'est pas un palindrome.

In [3]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
Entrez votre nombre : 7864
Votre nombre n'est pas un palindrome.

In [4]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
Entrez votre nombre : 112211
Votre nombre est un palindrome.
```

Figure 6

Calcul de la racine carrée par dichotomie

Exercice 4 : Calcul de la racine carrée par dichotomie

Dans cet exercice, on va écrire un programme Python qui calcule la racine carrée entière d'un nombre.

On utilise la méthode de dichotomie (ou recherche binaire), la même méthode qu'on a déjà utilisée en C.

```
# TP2 – Python : dichotomie
# Nom : HAMOUDA
# Prénom : Achraf
# Numéro Apogée : 23010848

a = int(input("Entrez un nombre entier positif : "))

bas = 0
haut = a
racine = 0

while bas <= haut:
    milieu = (bas + haut) // 2
    if milieu * milieu == a:
        racine = milieu
        break
    elif milieu * milieu < a:
        racine = milieu
        bas = milieu + 1
    else:
        haut = milieu - 1

print("La racine carrée entière de", a, "est :", racine)
```

Figure 7: code source

- La première chose qu'on a faite est de demander à l'utilisateur d'entrer un nombre entier positif, qu'on stocke dans la variable n.

- Ensuite, on a déclaré deux variables, bas = 0 et haut = n, pour définir l'intervalle de recherch où on va chercher la racine.
- On a aussi créé une variable racine = 0 pour garder la valeur trouvée.
- Après, on a utilisé une boucle while bas <= haut qui continue tant que l'intervalle n'est pas vide. À chaque tour, on calcule le milieu avec milieu = (bas + haut) // 2, Si milieu * milieu == n, on a trouvé la racine exacte. Si milieu * milieu < n, alors on déplace la borne basse (bas = milieu + 1). Sinon, on déplace la borne haute (haut = milieu - 1).
- Quand la boucle se termine, la variable racine contient la plus grande valeur entière telle que racine² ≤ n.

Exécution4 :

```
In [38]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
Entrez un nombre entier positif : 20
La racine carrée entière de 20 est : 4

In [39]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
Entrez un nombre entier positif : 36
La racine carrée entière de 36 est : 6

In [40]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
Entrez un nombre entier positif : 2
La racine carrée entière de 2 est : 1
```

Figure 8

Nombre parfait le plus proche

Exercice 5 : Nombre parfait le plus proche

Dans cet exercice, on va écrire un programme Python qui cherche le nombre parfait le plus proche d'un nombre donné par l'utilisateur.

Un nombre parfait est un nombre dont la somme de tous ses diviseurs propres (sauf lui-même) est égale au nombre lui-même.

```python
# TP2 – Python : Nombre parfait le plus proche
# Nom : HAMOUDA
# Prénom : Achraf
# Numéro Apogée : 23010848

n = int(input("Entrez un nombre entier : "))

def parf(x):
    somme = 0
    for i in range(1, x):
        if x % i == 0:
            somme += i
    return somme == x

plus_proche = n
distance = 0

while True:
    if parf(n - distance):
        plus_proche = n - distance
        break
    if parf(n + distance):
        plus_proche = n + distance
        break
    distance += 1

print(f"Le nombre parfait le plus proche de {n} est {plus_proche}")
```

Figure 9: code source

- D'abord, on a demandé à l'utilisateur d'entrer un nombre entier(n).

- Ensuite, on a créé une fonction parf() qui vérifie si un nombre est parfait : elle calcule la somme de tous les diviseurs du nombre x et retourne True si la somme est égale à x.
- Après, on a utilisé une boucle while True pour chercher le nombre parfait le plus proche du nombre donné.
- À chaque tour, on vérifie d'abord si (n - distance) est parfait, sinon on teste (n + distance).
- Dès qu'on trouve un nombre parfait, on le garde dans plus_proche et on arrête la boucle avec break.

Exécution5 :

```
In [63]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
Entrez un nombre entier : 22
Le nombre parfait le plus proche de 22 est 28

In [64]: %runfile C:/Users/achra/.spyder-py3/temp.py --wdir
Entrez un nombre entier : 1
Le nombre parfait le plus proche de 1 est 0
```

Figure 10

Fin